Compressed Computation is (probably) not Computation in Superposition

by **Jai Bhagat, Sara Molas Medina, Giorgi Giglemiani, StefanHex** 23rd Jun 2025

This research was completed during the Mentorship for Alignment Research Students (MARS 2.0) Supervised Program for Alignment Research (SPAR spring 2025) programs. The team was supervised by Stefan (Apollo Research). Jai and Sara were the primary contributors, Stefan contributed ideas, ran final experiments and helped writing the post. Giorgi contributed in the early phases of the project. All results can be replicated using this codebase.

Summary

We investigate the toy model of Compressed Computation (CC), introduced by Braun et al. (2025), which is a model that seemingly computes more non-linear functions (100 target ReLU functions) than it has ReLU neurons (50). Our results cast doubt on whether the mechanism behind this toy model is indeed computing more functions than it has neurons: We find that the model performance solely relies on noisy labels, and that its performance advantage compared to baselines diminishes with lower noise.

Specifically, we show that the Braun et al. (2025) setup can be split into two loss terms: the ReLU task and a noise term that mixes the different input features ("mixing matrix"). We isolate these terms, and show that the optimal total loss *increases* as we reduce the magnitude of the mixing matrix. This suggests that the loss advantage of the trained model does not originate from a clever algorithm to compute the ReLU functions in superposition (computation in superposition, CiS)

compute the ReLU functions in superposition (computation in superposition, CiS), but from taking advantage of the noise. Additionally, we find that the directions represented by the trained model mainly lie in the subspace of the positive eigenvalues of the mixing matrix, suggesting that this matrix determines the learned solution. Finally we present a non-trained model derived from the mixing matrix which improves upon previous baselines. This model exhibits a similar performance profile as the trained model, but does not match all its properties.

While we have not been able to fully reverse-engineer the CC model, this work reveals several key mechanisms behind the model. Our results suggest that CC is likely not a suitable toy model of CiS.

Introduction

Superposition in neural networks, introduced by Elhage et al. (2022), describes how neural networks can represent N sparse features in a $D < N$-dimensional embedding space. They also introduce *Computation in Superposition* (CiS) as computation performed entirely in superposition. The definition of CiS has been refined by Hänni et al. (2024) and Bushnaq & Mendel (2024)° as a neural network computing more non-linear functions than it has non-linearities, still assuming sparsity.

Braun et al. (2025) propose a concrete toy model of *Compressed Computation* (CC) which seemingly implements CiS: It computes 100 ReLU functions using only 50 neurons. Their model is trained to compute $y_i = x + ReLU(x_i)$ for $x_{0...99} \in [-1, 1]$. Naively one would expect an MSE loss per feature of 0.0833[1]; however the trained model can achieve a loss of ~ 0.06. Furthermore, an inspection of the model weights and performance shows that it does not privilege any features, suggesting it computes all 100 functions.[2]

Contributions: Our key results are that

1. The CC model's performance is dependent on noise in the form of a "mixing matrix" which mixes different features into the labels. It does not beat baselines without this mixing matrix.
2. The CC model's performance scales with the mixing matrix's magnitude (higher is better, up to some point). We also find that the trained model focuses on the top 50 singular vectors of the mixing matrix (all its neuron directions mostly fall into this subspace).
3. We introduce a new model derived from the SNMF (semi non-negative

matrix factorization) of the mixing matrix alone that achieves qualitatively and quantitatively similar loss to the trained model.

Methods

We simplify the residual network proposed by Braun et al. (2025), showing that the residual connection (with fixed, non-trainable embedding matrixes) serves as a source of noise that mixes the features. We show that a simple 1-layer MLP model trained on $\mathbf{y} = \mathrm{ReLU}(\mathbf{x}) + M\mathbf{x}$ produces the same results.

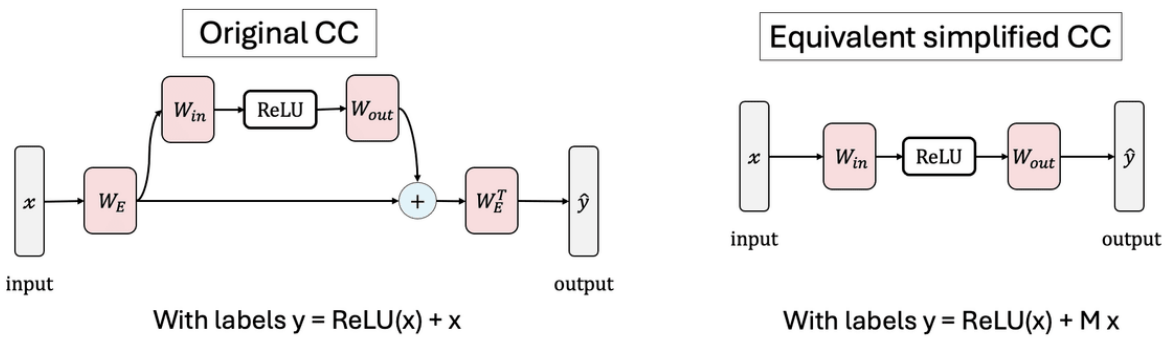


Figure 1: The original model architecture from Braun et al. (2025), and our simpler equivalent model. The labels for our (new) model are $y_i = \mathrm{ReLU}(x_i) + \sum_j M_{ij} x_i$. The matrix M mixes other input features x_j into the label y_i. Thus the MLP needs to learn both the ReLU term, and the mixing term.

We explore three settings for M:

1. The "embedding noise" case $M = 1 - W_E W_E^T$ is essentially equivalent to Braun et al. (2025).[3] The rows of W_E are random unit vectors, following Braun et al. (2025).
2. The "random noise" case where we set M to random values drawn from a normal distribution with mean 0 and standard deviation σ, typically between 0.01 and 0.05.
3. The "clean" case where we set $M = 0$.

The input vector is sparse: Each input feature x_i is independently drawn from a uniform distribution [-1, 1] with probability p, and otherwise set to zero. We also consider the maximally sparse case where exactly one input is nonzero in every sample. Following Braun et al. (2025) we use a batch size of 2048 and a learning rate of 0.003 with cosine scheduler. To control for training exposure we train all models for 10,000 non-empty batches. The only trainable parameters are W_{in} and W_{out} (not M); though in some cases (Figure 5a) we optimize σ together with the

weights.

Results

Qualitatively different solutions in sparse vs. dense input regimes

We reproduce the qualitative results of Braun et al. (2025) in our setting using $M = 1 - W_E W_E^T$ as well as a random matrix $M \sim N(0, 0.02)$. In both cases we find qualitatively different behaviour at high and low sparsities, as illustrated in Figure 2.

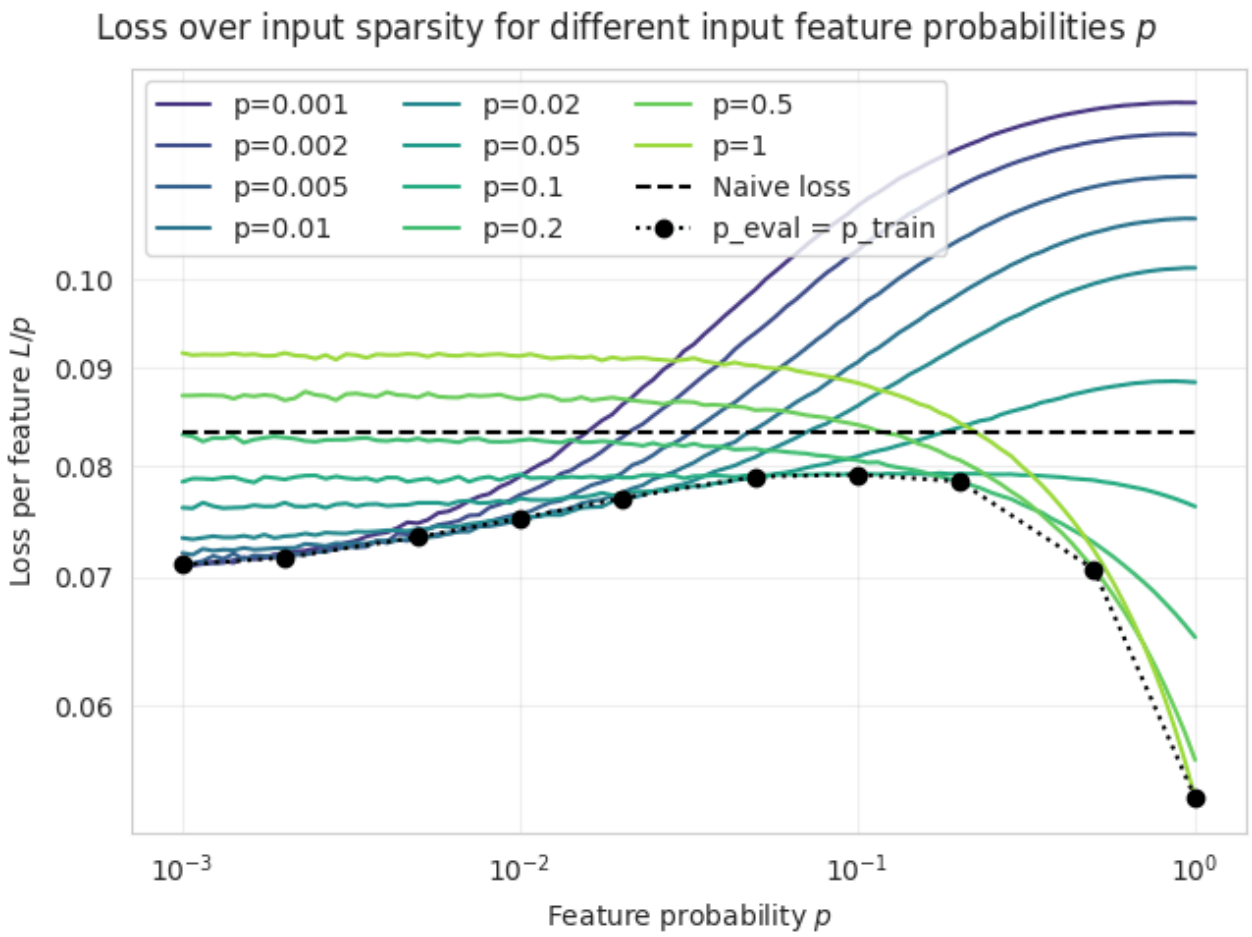


Figure 2: Loss per feature (L/p) as a function of evaluation sparsity. Each solid line corresponds to a model trained at a given sparsity. The models learn one of two solution types, depending on the input sparsity used during training: the "compressed computation" (CC) solution (violet) or a dense solution (green). Both types beat the naive baseline (dashed line) in their respective regime. Black circles connected by a dotted line represent the results seen by Braun et al. (2025), where models were evaluated only at their training sparsity.

1. In the sparse regime (low probability $p \lesssim 0.05$) we find solutions that perform well on sparse inputs, and less well on dense inputs. They typically

exhibit a similar input-output response for all features (Figure 3a), and weights distributed across all features (Figure 4a, equivalent to Figure 6 in Braun et al. 2025). The maximally sparse case (exactly one input active) behaves very similar to $p \leq 0.01$.

2. In the dense regime (high probability $p \gtrsim 0.2$) we find solutions with a constant per-feature loss on sparse inputs, but a better performance on dense inputs. These solutions tend to implement half the input features with a single neuron each, while ignoring the other half (Figures 3b and 4b).

Braun et al. (2025) studied the solutions trained in the sparse regime, referred to as Compressed Computation (CC).[4] In our analysis we will mostly focus on the CC model, but we provide an analysis of the dense model in this section°.

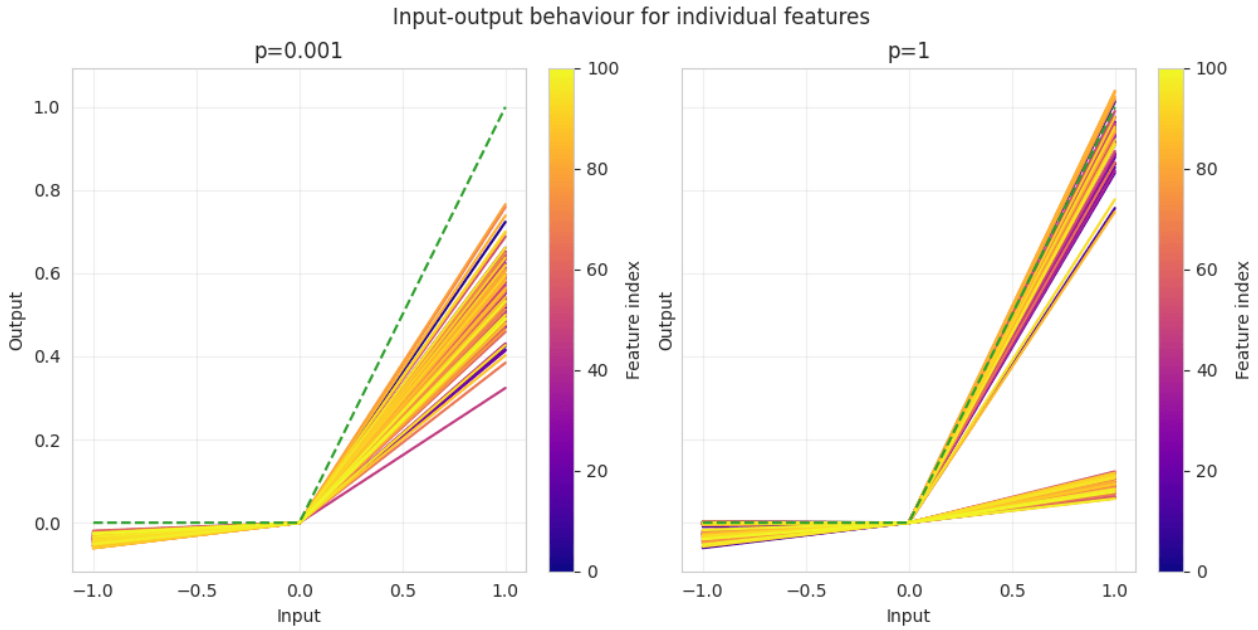


Figure 3: Input/output behaviour of the two model types (for one-hot inputs): In the "compressed computation" solution (left panel), all features are similarly-well represented: each input activates the corresponding output feature. In contrast, the dense solution (right panel) shows a strong (and more accurate) response for half the features, while barely responding to the other half. The green dashed line indicates the expected response under perfect performance.

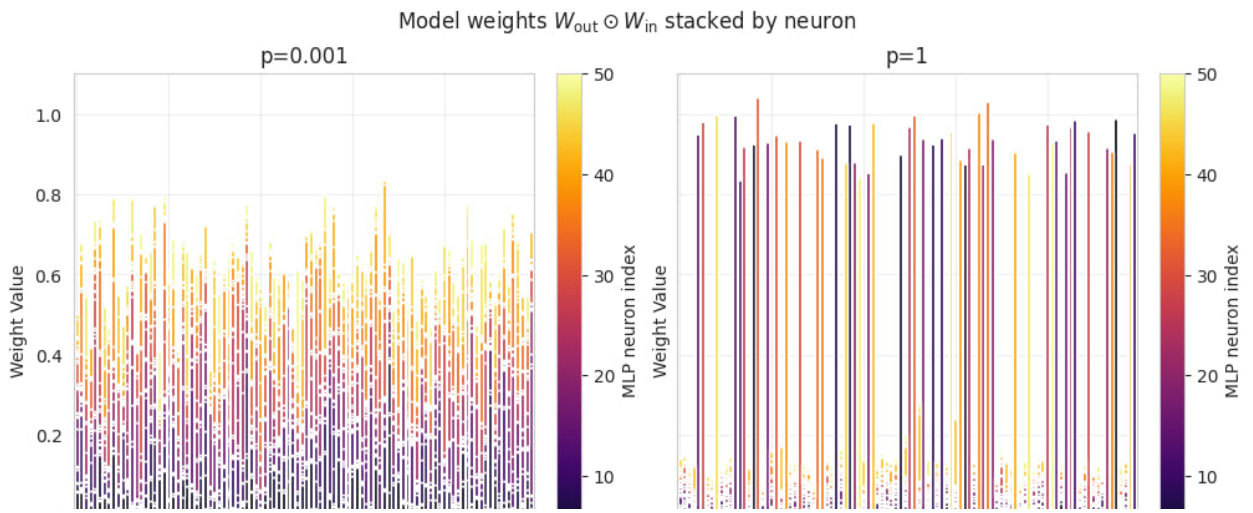


Figure 4: Weights representing each input feature, split by neuron. Each bar corresponds to a feature (x-axis) and shows the adjusted weight value from $W_{\text{out}} \odot W_{\text{in}}$, split by neuron index (color). The CC solution (left) combinations of neurons to represent each feature (to around 70%), whereas the dense solution (right) allocates a single neuron to fully (~100%) represent 50 out of 100 features.

Quantitative analysis of the Compressed Computation model

We quantitatively compare the model of Braun et al. (2025) with $M = 1 - W_E W_E^T$ to three simple mixing matrices: (a) a fully random mixing matrix M, (b) a random but symmetric M, and (c) $M = 0$.

We confirm that a fully random mixing matrix qualitatively reproduces the results (green curve in Figure 5a), and that a symmetric (but otherwise random) mixing matrix gives almost exactly the same quantitative results (red curve in Figure 5a) as the embedding case studied in Braun et al. (2025) would (blue line).[5][6]

Importantly, we find that a dataset without a mixing matrix does not beat the naive loss. Braun et al. (2025) hypothesized that the residual stream (equivalent to our mixing matrix) was necessary due to training dynamics but was not the source of the loss improvement. To test this hypothesis we perform two tests.

Firstly we train the CC models over a range of noise scales σ from 0 to 0.08, as shown in Figure 5b. We see that the loss almost linearly decreases with σ, for small σ. This is evidence that the loss advantage is indeed coming directly from the noise term.

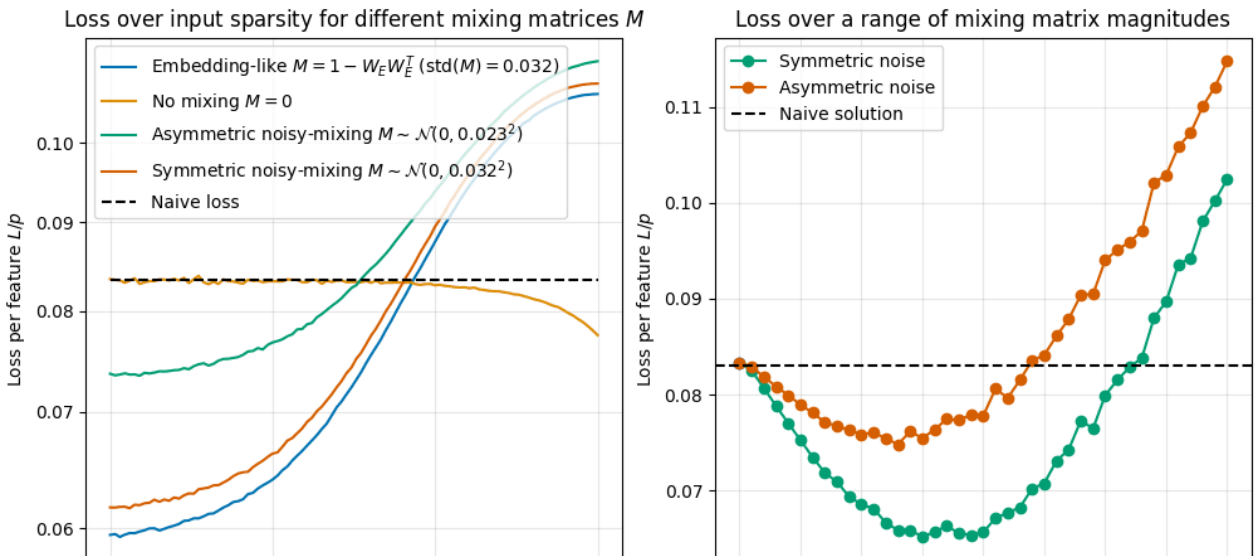


Figure 5, left: Loss per feature as a function of input sparsity, for different choices of M. We compare an embedding-like M (Braun et al. 2025, blue) to a fully random M (green) and a symmetric M (red) which is a random lower diagonal matrix mirrored; in both cases we set the magnitude if M that leads to the lowest loss. For comparison we also show a model trained on $M = 0$ (yellow). We find that all non-zero M lead to a qualitatively similar profile, and that a symmetrized random M almost gives the same result as Braun et al. (2025).
Right: Optimal loss as a function of mixing matrix magnitude σ (separately trained for every σ). For small σ the loss linearly decreases with the mixing matrix magnitude, suggesting the loss advantage over the naive solution stems from the mixing matrix M. At large values of σ, the loss increases again.

Secondly, to rule out training dynamics, we trained a CC model on a noisy dataset and then "transplanted" and fine-tuned it on the clean dataset (Figure 6). Here we find that the loss immediately rises to or above the naive loss when we switch to the clean dataset.

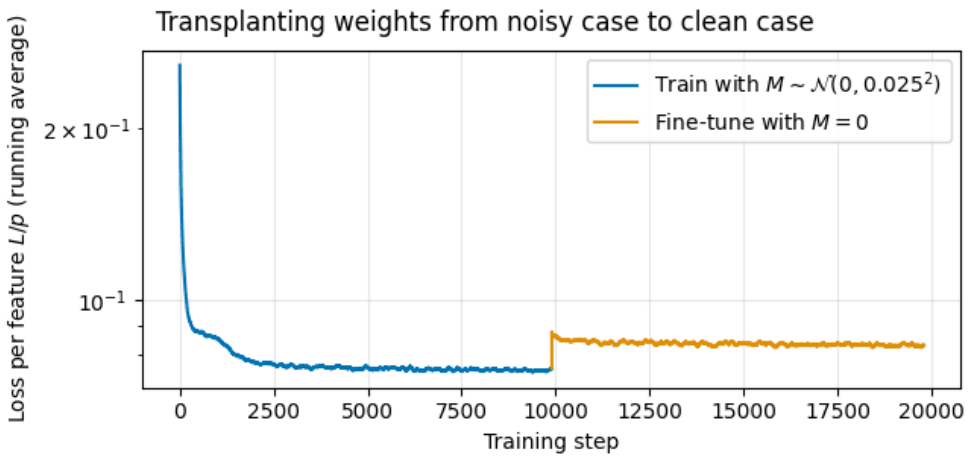


Figure 6: Training a model on the noisy dataset ($M \neq 0$), and then fine-tuning on the clean $M = 0$ case. We see that the loss jumps back up as soon as we switch to the clean task. This is evidence against the hypothesis that the CC solution wasn't learned on the clean case just due to training dynamics.

We conclude that compressed computation is entirely dependent on feature mixing.

Mechanism of the Compressed Computation model

Based on the previous results it seems clear that the CC model exploits properties of the mixing matrix M. We have some mechanistic evidence that the MLP weights are optimized for the mixing matrix M, but have not fully reverse-engineered the CC model. Additionally we design model weights based on the semi non-negative matrix factorization (SNMF) of $M + Identity$ which also beats

the naive loss but does not perform as well as the trained solution.

We find that the direction read (W_{in}) and written (W_{out}) by each MLP neuron lie mostly in the 50-dimensional subspace spanned by the positive eigenvalues of M (Figure 7a, top panels). We find this effect slightly stronger when taking the singular value decomposition (SVD) of $M + Identity$, [7] as shown in the bottom panels of Figure 7a. Additionally we measure the cosine similarity between singular- and eigenvectors projected through ($W_{\text{out}}W_{\text{in}}$) and find that only the top ~ 50 singular vectors are represented fully (Figure 7b).[8]

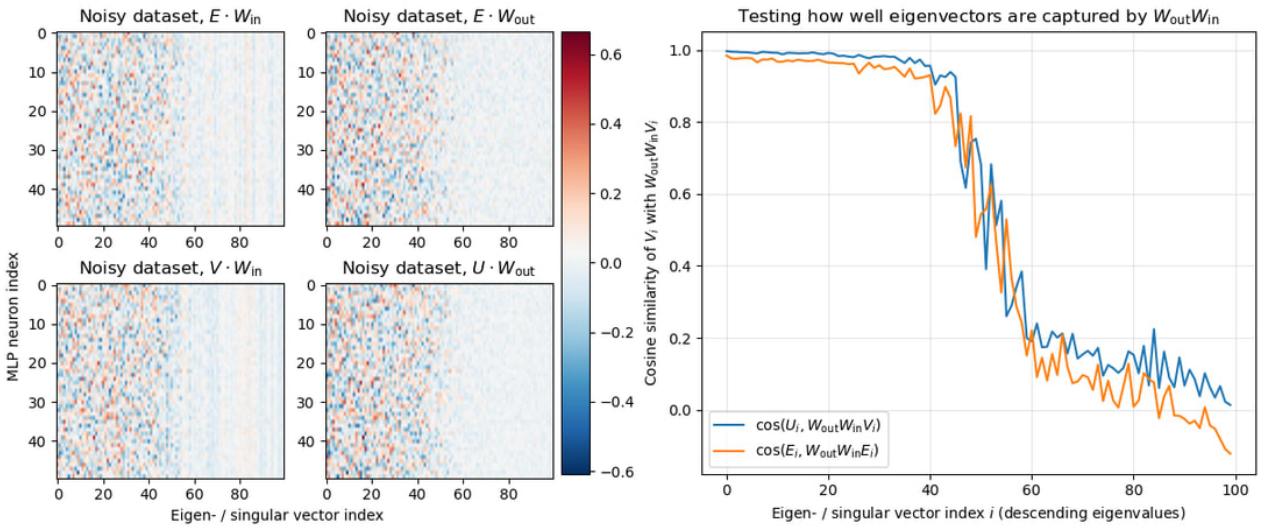
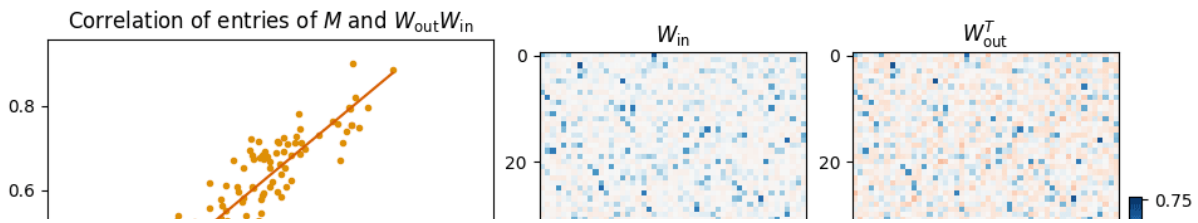


Figure 7, left: Cosine similarity between various eigen- and singular vectors (x-axis) and MLP neuron directions (y-axis). We show eigenvectors in the top panels, singular vectors in the bottom panels, the W_{in} matrix in the left panels, and the W_{out} matrix in the right panels. In all cases we see that the top-50 vectors (sorted by eigen- / singular value) have significant dot product with the neurons, while the remaining 50 vectors have near-zero dot products (black). Right: We test how well the ReLU-free MLP (i.e. just the $W_{\text{out}}W_{\text{in}}$ projection) preserves various eigen- (orange) and singular (blue) directions. Confirming the previous result, we find the cosine similarity between the vectors before and after the projection to be high for only the top 50 vectors.

We hypothesize that the MLP attempts to represent the mixing matrix M, as well as an identity component (to account for the ReLU part of the labels). We provide further suggestive evidence of this in the correlation between the entries of the M matrix, and the $W_{\text{out}}W_{\text{in}}$ matrix (Figure 8a): We find that the entries of both matrices are strongly correlated, and the diagonal entries of $W_{\text{out}}W_{\text{in}}$ are offset by a constant. We don't fully understand this relationship quantitatively though (we saw that the numbers depend on the noise scale, but it's not clear how exactly).



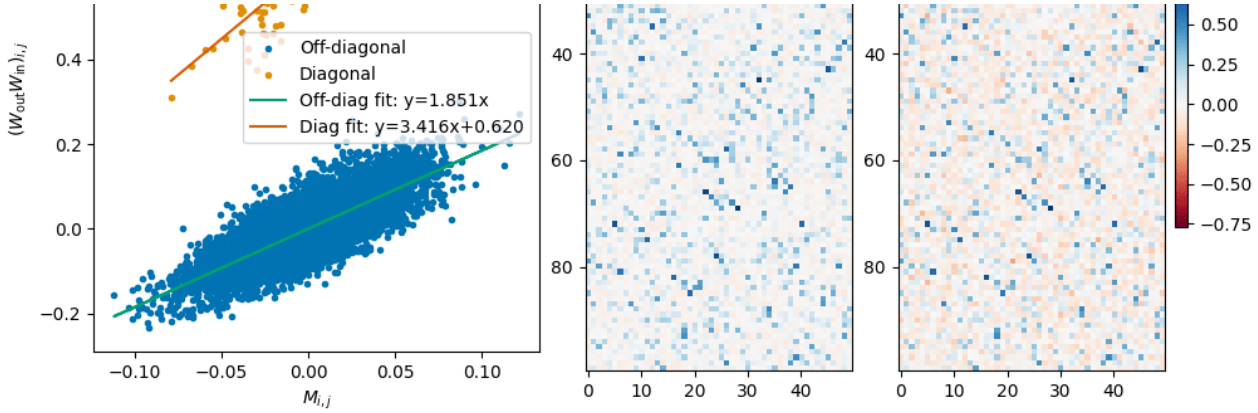


Figure 8, left: Scatter plot of the entries of the product $W_{\text{out}}W_{\text{in}}$ and the mixing matrix M; the entries are clearly correlated. The diagonal entries of $W_{\text{out}}W_{\text{in}}$ are offset by a constant, and seem to be correlated at a higher slope.
Right: Visualization of the MLP weight matrices W_{in} and W_{out}. We highlight that W_{in} is has mostly positive entries (this makes sense as it feeds into the ReLU), and both matrices have a small number of large entries.

The trained solution however is clearly not just an SVD with W_{in} and W_{out} determined by U and V. Specifically, as shown in Figure 8b, we note that W_{in} is almost non-negative (which makes sense, as it is followed by the ReLU).

Inspired by this we attempt to design a solution using the SNMF of M, setting W_{in} to the non-negative factor. Somewhat surprisingly, this actually beats the naive loss! Figure 9 shows the SNMF solution for different noise scales σ. The SNMF solution beats the naive loss for a range of σ values, though this range is smaller than the range in which the trained MLP beats the naive loss (Figure 5b). Furthermore, the SNMF solution does not capture all properties that we notice in W_{in} and W_{out}: The SNMF weights are less sparse, and the correlation between them and M (analogue to Figure 8) looks completely different (not shown in post, see Figure here).

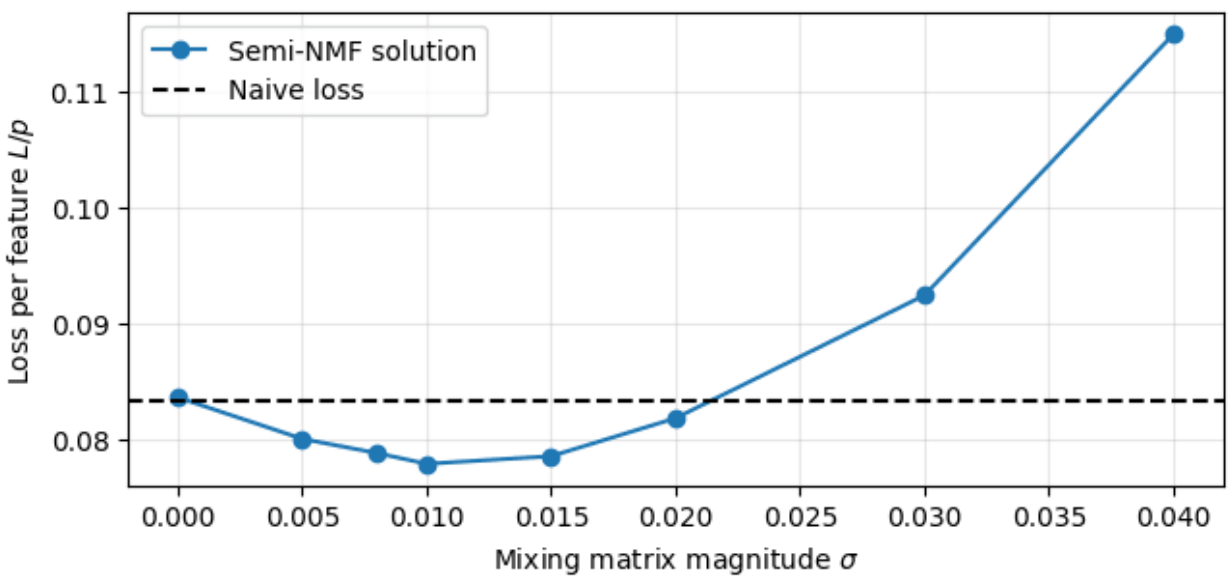


Figure 9: Loss of the SNMF solution, compared to the naive solution. Like in Figure 5b, the

Figure 9: Loss of the SNMF solution, compared to the naive solution. Like in Figure 5b, the solution does better than the naive loss for a range of σ values though the range is smaller and the loss is higher than for the trained model.

We conclude that we haven't fully understood how the learned solution works. However, we hope that this analysis shines some light on its mechanism. In particular, we hope that this analysis makes it clear that the initial assumption of CC being a model of CiS is likely wrong.

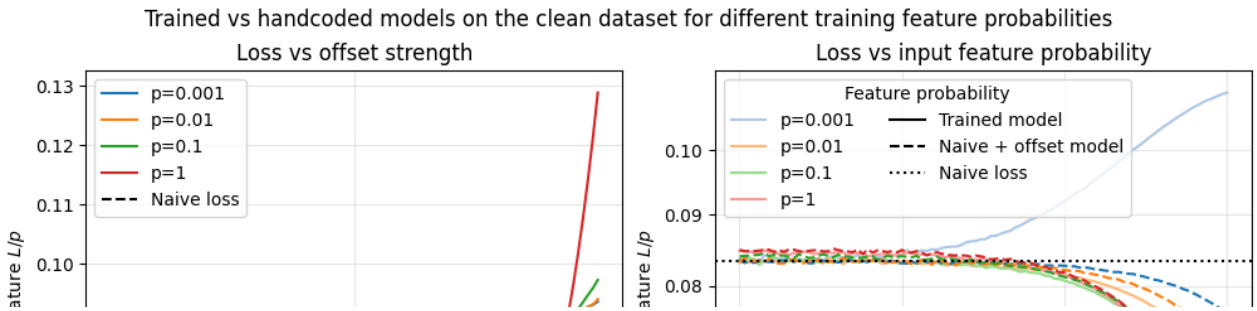
Mechanism of the dense solution

We now return to the solution the model finds in the high feature probability (dense) input regime. As shown in Figure 2, models trained on dense inputs outperform the naive baseline on dense inputs. Revisiting Figures 3b and 4b, we notice an intriguing pattern: roughly half of the features are well-learned, while the other half are only weakly represented.

Our hypothesis is that the model represents half the features correctly, and approximates the other half by emulating a bias term. Our architecture does not include biases, but we think the model can create an offset in the outputs by setting the corresponding output weight rows to positive values, averaging over all features. This essentially uses the other features to, on average, create an offset.

We test this "naive + offset" solution on the clean dataset ($M = 0$) as the behaviour seems to be the same regardless of noise (but we did not explore this further), and find that the hardcoded naive + offset solution does in fact match the trained models' losses. Figure 10a shows the optimal weight value (same value for every W_{out} entry of non-represented features), and Figure 10b shows the corresponding loss as a function of feature probability. We find that the hardcoded models (dashed lines) closely match or exceed the trained models (solid lines).

We thus conclude that the high density behaviour can be explained by a simple bias term, and not particularly interesting. We have not explored this case with a non-zero mixing matrix but we expect that one could generalize the naive + offset solution to noisy cases.



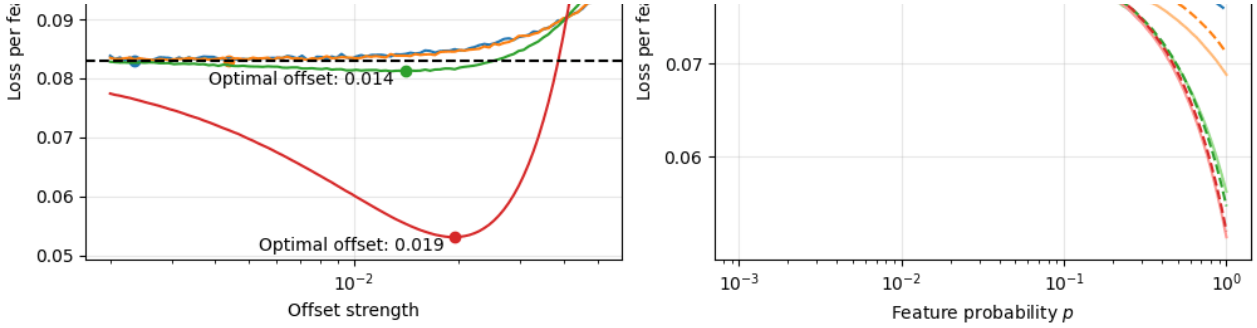


Figure 10, left: A non-zero offset in the W_{out} entries of unrepresented features improves the loss in the dense regime. We determine the optimal value empirically for each input feature probability p.
Right: This hand-coded naive + offset model (dashed lines) consistently matches or outperforms the model trained on clean labels (solid lines) in the dense regime. (Note that this plot only shows the clean dataset ($M = 0$) which is why no solution outperforms the naive loss in the sparse regime.)

At high training feature probabilities, tuning the offset value in the naive + offset model significantly improves performance beyond the naive baseline. However, when the feature probability is 0.1 or lower, no offset leads to better-than-naive performance. This further supports the idea that the model adopts one of two distinct strategies depending on the feature probability encountered during training.

Discussion

Our work sheds light on the mechanism behind the Braun et al. (2025) model of compressed computation. We conclusively ruled out the hypothesis that the embedding noise was only required for training, and showed that the resulting mixing matrix is instead the central component that allows models to beat the naive loss.

That said, we have not fully reverse-engineered the compressed computation model. We would be excited to hear about future work reverse engineering this model (please leave a comment or DM / email StefanHex!). We think an analysis of how the solution relates to the eigenvectors of M, and how it changes for different choices of M are very promising.[9]

Acknowledgements: We want to thank @Lucius Bushnaq and @Lee Sharkey for giving feedback at various stages of this project. We also thank @Adam Newgas, @jake_mendel, @Dmitrii Krasheninnikov, @Dmitry Vaintrob, and @Linda Linsefors for helpful discussions.

1. ^ This loss corresponds to implementing half the features, ignoring the other half. The number is derived from integrating $\mathrm{ReLU}(x)$.
2. ^ We're using the terms feature and function interchangeably, both refer to the 100 inputs and their representation in the model.
3. ^ Our W_{in}/W_{out} are just 100x50 dimensional, while Braun et al. (2025)'s are 1000x50 dimensional due to the embedding, but this does not make a difference.
4. ^ Braun et al. (2025) say "We suspect our model's solutions to this task might not depend on the sparsity of inputs as much as would be expected, potentially making 'compressed computation' and 'computation in superposition' subtly distinct phenomena". We now know that this was due to evaluating a model's performance only at its training feature probability, rather than across a set of evaluation feature probabilities, i.e. they measured the black dotted line in Figure 2.
5. ^ For Figure 5 we chose the optimal noise scale for each model, which happens to be close to the standard deviation of the mixing matrix corresponding to the embeddings chosen in Braun et al. (2025). This was coincidental as Braun et al. (2025) chose $d_{embed} = 1000$ for unrelated reasons.
6. ^ Two further differences between Braun et al. (2025)'s embed-like case and the symmetrized case are (a) the diagonal of M is zero in the embed-like case, and (b) the entries of M are not completely independent in the embed-like case as they are all derived by different combinations of the embedding matrix entries. We find that setting the diagonal of the random M to zero almost matches the low feature probability loss of the embed-like case, though it slightly increases the high-p loss.
7. ^ The eigenvalues of and the singular values of $M + Identity$ are approximately related.
8. ^ This is not due to M being low-rank; the singular value spectrum is smooth (as expected from a random matrix).
9. ^ During our experiments we found that the loss depends on the rank of M, though we found that, depending on the setting, reducing the rank could either increase or decrease the loss.

Mentioned in

186 Shallow review of technical AI safety, 2025

73 Circuits in Superposition 2: Now with Less Wrong Math

9 comments, sorted by top scoring

[-] **Lucius Bushnaq** 9mo ▼ 5 ▲ ✕ 3 ✓

Thank you for looking into this.

This investigation updated me more toward thinking that Computation in Superposition is unlikely to train in this kind of setup, because it's mainly concerned with minimising *worst-case* noise. It does lots of things, but it does them all with low precision. A task where the model is scored on how close to correct it gets many continuously-valued labels, as scored by MSE loss, is not good for this.

I think we need a task where the labels are somehow more discrete, or the loss function punishes outlier errors more, or the computation has multiple steps, where later steps in the computation depend on lots of intermediary results computed to low precision.

[-] **Dan Braun** 9mo * ▼ 3 ▲ ✕ 2 ✓

I think this is a fun and (initially) counterintuitive result. I'll try to frame things as it works in my head, it might help people understand the weirdness.

The task of the residual MLP (labelled CC Model here) is to solve y = x + ReLU(x). Consider the problem from the MLP's perspective. You might think that the problem for the MLP is to just learn how to compute ReLU(x) for 100 input features with only 50 neurons. But given that we have this random W_E matrix, the task is actually more complicated. Not only does the MLP have to compute ReLU(x), it also has to make up for the mess caused by $W_E W_E^T$ not being an identity.

But it turns out that making up for this mess actually makes the problem easier!

1

[-] **Jai Bhagat** 9mo ▼ 1 ▲ ✕ 0 ✓

Yes! But only if the mess is the residual stream, i.e. includes x! This is the heart of the necessary "feature mixing" we discuss

[-] **Linda Linsefors** 9mo * ▼ 2 ▲ ✕ 0 ✓

I did some quick calculations for what the mse per feature should be for compressed storage. I.e. storing T features in D dimension where T > D.

I assume every feature is on with probably p. On feature equals 1, off feature equals 0. Mse is mean square error for linear readout of features.

For random embeddings (super possition):

mse_r = Tp/(T+D)

If using the D neurons to embed T features exactly, and output feature value constant p, for the rest.

mse_d = p(1-p)(T-D)/T

This suggest we should see a transition between these types of embeddings when

mse_r = mse_d when T^2/D^2=(p-1)/p

For T=100 and D=50, this means p=0.2

The model in this post is doing a bit more than just embedding features. But I don't think, it can't do better than the most effective embedding of the T output features in the D neurons?

mse_r only depends on E[(u dot v)^2]=1/D where v and u are diffrent embedding vectors. Lots of ebeddings have this property, e.g. embedding features along random basis vectors, i.e. assigning each feature to a random single neuron. This will result in some embedding vectors being exactly identical. But the mse (L2) loss is equally happy with this as with random (almost orthogonal) feature directions.

[-] **Linda Linsefors** 9mo 2 0

Does p=1 mean that all features are always on?

If yes, how did it fail to get perfect loss in this case?

[-] **StefanHex** 9mo 5 0

The features are on, but arbitrary values between -1 and 1 (I assume you were thinking of the binary case)

[-] **Linda Linsefors** 9mo 2 0

Yes, thanks!

[-] **TheManxLoiner** 9mo 1 0

Is code for experiments open source?

[-] **StefanHex** 9mo 3 0

Yep, the main code is in this folder! You can find

- the main logic in `mlpinsoup.py`
- notebooks reproducing the plots for each result section in the `nb{1,2,3,4}...` files

Moderation Log

Curated and popular this week

258	On The Independence Axiom	Ihor Kendiukhov	2d	55
154	Economic efficiency often undermines sociopolitical au...	Richard_Ngo	6d	33
130	My Willing Complicity In "Human Rights Abuse"	AlphaAndOmega	1d	11